1. Imagine you're driving on a long road trip, but instead of focusing on the road, you're too busy trying to remember the classic formula: Distance = Speed × Time. To make life easier (and to avoid any unnecessary mental gymnastics while driving), let's create two C++ classes—Speed and Time—to store speed in km/h and time in hours, respectively. Now, because these two are a bit shy and like to keep their data private, we'll introduce them to a mutual friend, a friend function called calculateDistance(), which will sneakily access their private members and compute the total distance traveled. The goal? Write a C++ program where the calculateDistance() function takes values from both classes (say, a speed of 60 km/h and a duration of 2 hours) and calculates how far you've traveled. Finally, in the main() function, instantiate objects with these values and print out the result—because let's be honest, no one wants to manually do the math while on a road trip! 🚗💨

2. Cricket fans, get ready for some coding action! Imagine the ICC is developing a high-tech system to analyze a batsman's total runs in a match. To make this happen, let's create two C++ classes—Runs and BallsFaced—to store the number of runs scored and the number of balls faced by a batsman, respectively. But since these two classes are as secretive as a team's match strategy, we'll introduce them to a mutual friend function called calculateStrikeRate(), which will access their private data and calculate the player's strike rate using the formula:
   Strike Rate = (Runs / Balls Faced) × 100
   Your task is to write a C++ program where calculateStrikeRate() takes values from both classes (e.g., a batsman scores 85 runs off 60 balls) and computes the strike rate. Finally, in the main() function, instantiate objects with sample values and print out the calculated strike rate—because even the ICC needs a little coding magic to keep up with the game! 🏏🔥

3. A C++ developer, utterly fed up with the simplicity of real numbers, decides to shake things up by creating a ComplexNumber class—because why settle for numbers that behave when you can have ones with imaginary side effects? But just storing complex numbers isn't enough—these numbers need action and operator overloading to truly unleash their potential! First up is addition (+), where two numbers team up like superheroes—3 + 4i and 1 + 2i join forces to become 4 + 6i. Then comes subtraction (-), for when numbers need a little personal space—like 8 + 3i kicking 5 + 6i to the curb, leaving behind a 3 - 3i situation. Of course, there's multiplication (*), where two numbers combine with chaotic consequences—(2 + 3i) * (1 + 4i) suddenly transforms into -10 + 11i, proving that multiplying complex numbers is anything but simple. Finally, there's the equality check (==), because some numbers are identical (3 + 5i == 3 + 5i), while others are just wannabes (3 + 5i != 4 + 5i—nice try). Your mission? Write a ComplexNumber class in C++ that makes all this mathematical madness possible using operator overloading—because even in C++, numbers shouldn't get a free pass to be boring!

4. Traffic jams are the ultimate test of patience, but what if C++ could take control of the chaos? Enter the TrafficSystem class, designed to bring order to the madness using operator overloading. First up is addition (+), because when more cars enter a busy intersection (50 cars + 20 more), the total count surges to 70, making the situation even worse. Then comes subtraction (-), offering a glimmer of hope—if 15 cars finally move out, the congestion eases to 55, though not for long! Next, there's multiplication (*), useful for scenarios like a traffic signal failure doubling the wait time—if the delay was 5 minutes, it instantly jumps to 10 minutes, leaving drivers questioning all their life choices. And finally, there's equality (==), critical for detecting traffic gridlocks—when road A and road B both have 100 cars stuck motionless, it's official: the city is doomed! Your mission? Write a TrafficSystem class in C++ that smartly handles all these scenarios with operator overloading, ensuring that while real-world traffic stays frustrating, at least the code stays efficient and under control! 🚦🚗💨

5. Meet Student, a class that holds a student's name and their three precious grades—because what's school without some academic drama? But wait, students can't just update their own grades (obviously), so we bring in Teacher, a class with special access privileges (a.k.a. a friend class) that can peek into grades, modify them, and perhaps even save students from academic disaster! To keep things fair and scientific, there's also a friend function called calculateAverageGrade() that computes the average grade—because math doesn't lie (even if students wish it would). In main(), we'll simulate a high-stakes grading scenario: first, we'll display the student's grades, then let the teacher graciously update one of them (whether out of generosity or mercy, who knows?), and finally, we'll recalculate the average to see if things improved or got even worse. Ready to code this virtual school system? Just remember: Teacher holds all the power! 🎓📚😊